EventEmitter

10 min read · Updated 24 Jul 2026 · Topic 20 of 70

Bookmark · Mark done

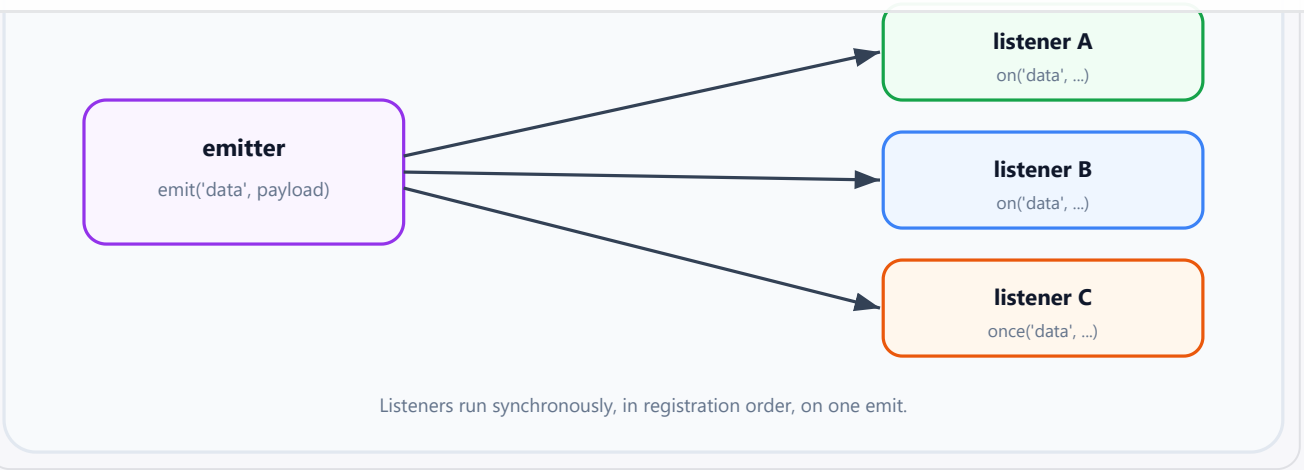
Why it matters

Promises and `async/await` model a *single* future result. But a huge amount of Node is about things that happen *repeatedly* over time: a socket receiving many chunks, an HTTP server handling request after request, a stream emitting `data` then `end`, a file watcher firing on every change. A promise can't model "this will happen many times." That's what the **EventEmitter** is for, and it sits at the heart of Node's most important subsystems — streams, servers, sockets, the process object itself.

Understanding `EventEmitter` is what lets you read Node's own APIs fluently (`req.on('data', …)`, `server.on('connection', …)`) and build clean decoupled architectures of your own. It's also a classic interview topic: the memory-leak warning about too many listeners, the difference between `on` and `once`, the fact that emission is synchronous, and the special status of the `'error'` event all trip up people who've only ever used it casually.

The concept, simply

An `EventEmitter` is an object that lets you **publish** named events and **subscribe** to them — the observer (pub/sub) pattern, built into Node's core. You register a listener with `emitter.on('event', handler)`, and later something calls `emitter.emit('event', …args)`, which invokes every registered handler for that name, in order, passing along the arguments. The emitter and the listeners don't know about each other's internals — that decoupling is the whole point.



A single emit call invokes every registered listener for that event synchronously, in the order they were added.

Two subscription verbs matter: `on` (alias `addListener`) registers a handler that fires *every* time the event is emitted; `once` registers one that fires *only the first time* and then removes itself automatically. You unsubscribe with `off` (alias `removeListener`). Most Node objects you use daily — streams, servers, sockets — *are* EventEmitters or inherit from one.

How it works

Internally an emitter keeps a map from event name to an array of listener functions. `emit(name, ...args)` looks up that array and calls each function **synchronously**, one after another, on the current stack — right there, before `emit` returns. This surprises people: emitting an event is *not* asynchronous. If a listener does slow synchronous work, it blocks. If a listener throws, it propagates out of the `emit` call and can stop later listeners from running. Listeners fire in registration order; `emit` returns `true` if there were listeners, `false` if none.

`once` works by wrapping your handler: the wrapper removes itself from the listener array before (or as) it calls your function, guaranteeing a single invocation. `removeListener` / `off` splices the function out of the array — which is why you must pass the *same function reference* you registered; an inline arrow can't be removed because you no longer hold its reference.

Two special behaviours are worth memorising. First, the `'error'` event is privileged: if an emitter emits `'error'` and there is **no** listener for it, Node *throws* the error, typically

crashing the process. So any long-lived emitter must have an `'error'` listener. Second, there's a built-in leak guard: if you add more than `defaultMaxListeners` (10) listeners for a single event, Node prints a `MaxListenersExceededWarning`. It's not an error — it's a heuristic that you're probably adding listeners in a loop and forgetting to remove them.

It helps to contrast the EventEmitter with a promise directly, because they solve genuinely different shapes of problem. A promise is a *value*: it settles exactly once, it caches its result, and a consumer that subscribes *after* it settles still gets the value. An event is a *notification*: `emit` fans out to whoever is listening *at that moment*, keeps no history, and a listener that subscribes after the emission simply misses it. That single difference — replayable value versus fire-and-forget signal — is the deciding factor when you're choosing between them. Repeated occurrences, multiple independent subscribers, or "tell me each time X happens" all point to events; "give me the one result of this operation" points to a promise.

A few more mechanics round out the model. `emitter.listenerCount(name)` tells you how many handlers are registered, and `emitter.eventNames()` lists the events that currently have any. `prependListener` adds a handler to the *front* of the array so it runs before existing ones — occasionally useful for instrumentation. `emitter.setMaxListeners(n)` raises the warning threshold for that instance, and `EventEmitter.defaultMaxListeners` changes it globally. And because listeners run synchronously, the value `emit` returns (`true` if any listener ran, else `false`) is known immediately — there is no waiting involved.

The core method surface is small; this table is effectively the whole working vocabulary:

Method	Effect
`on(name, fn)` / `addListener`	Subscribe; fires on every emission
`once(name, fn)`	Subscribe; fires once, then self-removes
`off(name, fn)` / `removeListener`	Unsubscribe by function reference
`emit(name, ...args)`	Invoke every listener synchronously; returns `true` if any ran

`prependListener(name, fn)`	Subscribe at the front of the queue
`removeAllListeners(name?)`	Drop all listeners for one event, or for every event
`listenerCount(name)` / `eventNames()`	Introspect current subscriptions
`setMaxListeners(n)`	Raise this instance's leak-warning threshold

Because listeners run synchronously and share the emitter's stack, a listener that `await`s does *not* make `emit` wait: `emit` returns the instant the synchronous portion of every listener has run, long before any async work inside them settles. The EventEmitter contract simply has no way to report "all listener work finished" — if you need that, collect the listeners' promises yourself or model the flow with a promise instead. The same shared stack is why a throw inside one listener can stop the listeners registered after it from running at all: the exception unwinds `emit` before it reaches them. Keeping each listener fast, non-throwing, and self-contained is therefore not just style — it is what keeps one subscriber from silently breaking the others.

Code examples

Basic pub/sub with `on`, `once`, and `emit`. Note the once-listener fires a single time:

JAVASCRIPT

Copy

```
const { EventEmitter } = require('node:events');
const bus = new EventEmitter();

bus.on('order', (id, total) => console.log('order', id, 'for', total));
bus.once('ready', () => console.log('ready - fires only once'));

bus.emit('ready');           // logs
bus.emit('ready');           // nothing - once already removed itself
bus.emit('order', 42, 999);  // logs: order 42 for 999
```

Building your own emitter by extending the class — the idiomatic way to give an object an event API:

JAVASCRIPT Copy

```
class Job extends EventEmitter {
  async run() {
    this.emit('start');
    try {
      const result = await doWork();
      this.emit('done', result); // listeners run synchronously here
    } catch (err) {
      this.emit('error', err);   // MUST have an 'error' listener
    }
  }
}

const job = new Job();
job.on('start', () => console.log('started'));
job.on('done', (r) => console.log('finished', r));
job.on('error', (e) => console.error('failed', e.message)); // required
job.run();
```

Removing listeners correctly (keep the reference) and proving emission is synchronous:

JAVASCRIPT Copy

```
function onData(chunk) { console.log('got', chunk); }
bus.on('data', onData);
bus.off('data', onData);      // works — same reference
// bus.off('data', (c) => {}); // would NOT remove anything

console.log('before emit');
bus.on('sync', () => console.log('during emit'));
bus.emit('sync');
console.log('after emit');
// Order: before emit → during emit → after emit  (synchronous!)
```

Bridging an event-based flow to a promise with `events.once` — useful when you want to `await` the next occurrence of an event:

```
const { once } = require('node:events');

async function waitForConnect(server) {
  server.listen(3000);
  await once(server, 'listening'); // resolves on the event, rejects on 'e
  console.log('server is up');
}
```

Common mistakes & gotchas

- **No `'error'` listener.** An emitted `'error'` with no handler is *thrown* and usually crashes the process. Always attach an `'error'` listener to long-lived emitters (streams, sockets, servers).
- **Assuming `emit` is async.** Listeners run synchronously on the current stack before `emit` returns. Slow listener work blocks; if you need async, defer inside the listener.
- **The memory leak: adding listeners in a loop.** Registering a listener per request/iteration without removing them grows the array unboundedly. The `MaxListenersExceededWarning` at 11 listeners is Node telling you so.
- **Removing an anonymous listener.** You can only `off` / `removeListener` a function you hold a reference to. Inline arrows can never be removed — keep a named reference or use `once`.
- **A throwing listener halting the rest.** An exception in one synchronous listener propagates out of `emit` and can prevent later listeners from running. Keep listeners defensive.
- **Bumping `setMaxListeners` to silence the warning.** That hides a real leak. Fix the registration/removal logic instead, and only raise the limit when you genuinely need many listeners.
- **Using events for one-shot request/response.** If something happens exactly once and you need its value, a promise is clearer than wiring up and tearing down listeners.

Best practices

- Always register an `'error'` listener on any emitter that can emit one; unhandled `'error'` crashes the process.
- Use `once` for one-time events so the listener cleans itself up automatically.
- Keep named references to listeners you'll need to remove, and remove them when the subscriber goes away to prevent leaks.
- Keep listeners fast and non-throwing; do heavy or async work by scheduling it, not inline in the listener.
- Treat the `MaxListenersExceededWarning` as a leak signal to investigate, not noise to suppress.
- Reach for `events.once(emitter, name)` to `await` a single future event, and `events.on` (async iterator) to consume a stream of them.
- Choose events for *recurring* or *broadcast* notifications; choose promises for a *single* eventual result.

⭐ Interview lens

Q: Is `emit` synchronous or asynchronous?
Synchronous. It invokes every listener for the event in registration order, on the current stack, before `emit` returns. This surprises people who assume events are always deferred.

Q: What happens if you emit `'error'` with no listener? (trap)
Node throws the error as an exception, which typically crashes the process. The `'error'` event is special-cased precisely so silent failures don't go unnoticed — always attach a handler.

Q: Difference between `on` and `once`?
`on` fires the listener on every emission; `once` fires it only the first time and then auto-removes it. `once` is implemented as a self-removing wrapper around your handler.

Adding more than 10 listeners (the default) for one event name. It's a warning, not an error — a heuristic that you're likely leaking listeners by adding them repeatedly without removing them.

Q: Why can't you remove an inline arrow-function listener?

`removeListener` / `off` matches by function reference. An inline arrow creates a new function you don't keep a handle to, so there's nothing to pass back to remove it. Keep a named reference.

Q: When would you use an EventEmitter over a promise?

When the thing happens more than once or is broadcast to multiple independent subscribers — streaming data, connection events, progress updates. Promises model a single eventual value; events model an ongoing stream of occurrences.

Cheat-card

- **Pattern:** pub/sub — `on` / `once` to subscribe, `emit` to publish.
- `emit` **is synchronous** — listeners run in order, on the current stack.
- `once` fires once and self-removes; `off` needs the same reference.
- `'error'` with no listener throws and crashes — always handle it.
- **Leak guard:** >10 listeners on one event → warning; fix removal, don't mute.
- **Use events** for many/broadcast occurrences; **promises** for a single result.

QUICK CHECK — no marks, just for you.

1. Is `emitter.emit(name, ...args)` synchronous or asynchronous?

- ◯ Asynchronous — listeners run on the next tick

- It depends on the number of listeners
- Synchronous — every listener runs in registration order on the current stack before emit returns

Check answer

2. What happens if an emitter emits `error` with no listener registered for it?

- Node throws the error, typically crashing the process
- The error is silently ignored
- Node retries the emission later
- The error is queued until a listener is added

Check answer

‹ Previous
Error Handling in Async Code

Next ›
The fs Module: Reading & Writing Files

LEARN MORE

ARTICLES & DOCS

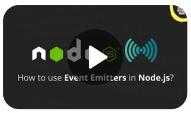
Events (EventEmitter) API
nodejs.org

VIDEOS



Node JS Tutorial for Beginners #8 - The Node Event Emitter

Net Ninja



Node.js "Event Emitters" Explained

Software Developer Diaries

Hand-picked references for this lesson.